

Debugging for Scala Control Flow DSLs

Finn Hackett and Ivan Beschastnikh

1 Motivation

One strength of Scala is how convenient it is for building Domain-Specific Languages (DSLs). Many Scala libraries involve some kind of DSL, and a popular application of DSLs is to offer non-default forms of control flow to the developer. We call this sort of DSL a *control flow DSL*.

For example, I/O effects, both in the form of standard library futures and beyond [12, 15], offer diverse flavors of concurrent task management, adapted to a relatively direct syntax using combinators and for-notation. Apache Spark [14] applies this concept to distributed data processing. Parsing [5, 8], with its repeated choices and backtracking, is also a popular application of control flow DSLs. Utilities like the standard library’s TailCalls [9], or the Cats Eval monad [13], allow developers to write mutually tail-recursive methods without the risk of causing stack overflows [1]. While these control flow DSLs are a powerful way of abstracting away control flow concerns, they make one thing much harder: debugging.

The key feature of control flow DSLs, programmatically replacing the relationship between program logic and the JVM callstack, is also the reason behind why debugging such programs is a significant challenge. Common debugging approaches, such as looking at the stacktrace from a program crash, or attaching a debugger, no longer work since the corresponding artifacts do not accurately represent the program’s logical state.

In this talk we propose a practical way to address this design weakness, by bringing a useful debugging experience to control flow DSLs. We first illustrate the problem in detail (Section 2), and then describe our prototype solution, a lightweight in-process implementation of the Debug Adapter Protocol for DSLs (Section 3). To make our proposal precise, we also include a prospective talk outline (Section 5).

2 Demonstrating the Problem

To give a concise but contrived example, consider Listing 1. The Listing shows two functions, `fn1` and `fn2`. The illustrative algorithm just counts down from any integer to 0 with some special cases. Written as-is using conventional recursion, this program has a problem. Given a sufficiently big input, it will crash with a stack overflow. While Scala provides some support for simple tail-call elimination, it only works with directly recursive functions, and our example isn’t tail recursive anyway.

To work around the possibility of a stack overflow, Listing 2 shows how to rewrite our program using TailCalls, a utility from the Scala standard library. Rather than directly compute the result, normal control flow is replaced by `map`

```
def fn1(n: Int): Int =
  n match
    case 0 => -1
    case 42 => fn2() + 2
    case _ => fn1(n - 1) + 1

def fn2(): Int = ???

@main
def run(): Unit =
  println(fn1(50))
```

Listing 1. A recursive integer count-down program with a crashing edge case.

```
import scala.util.control.TailCalls.*

def fn1(n: Int): TailRec[Int] =
  n match
    case 0 => done(-1)
    case 42 =>
      tailcall(fn2())
        .map(_ + 2)
    case _ =>
      tailcall(fn1(n - 1))
        .map(_ + 1)

def fn2(): TailRec[Int] = tailcall(???)

@main
def run(): Unit =
  println(fn1(50).result)
```

Listing 2. Stack overflow-free version of Listing 1.

and `flatMap` (with or without syntax sugar like for comprehensions). Most of the program returns objects representing future computations, all of which are run from the `TailRec.result` method. That result-calculating method implements its own virtual control flow, in this case with a heap-based stack that supports recursion as deep as the JVM’s heap will allow, while also performing full runtime tail call elimination where possible. This setup allows us to write the program how we want, while relying on a custom notion of control flow to also run it how we want.

The debugging challenge with the new solution comes up when we run our program, see that it crashes, and want to triage the issue. Running with TailCalls, we see a stack-trace like the one in Listing 3. It points to where the exception

```
Exception NotImplementedError
  at fn2
  at TailRec.result
  at main
```

Listing 3. Stacktrace from running Listing 2, edited for presentation.

```
Exception NotImplementedError
  at fn2
  at fn1
  ...
  at fn1
  at main
```

Listing 4. Stacktrace from running Listing 1, edited for presentation.

came from, but the only other information is that our exception came from somewhere in `TailRec.result`¹. The same thing happens when we run our debugger: we see that `fn2` was called from somewhere at the exception throw site, and single-stepping shows us a difficult to parse mix of lambdas and `TailCalls` implementation code.

To illustrate what is missing, consider if we ran Listing 1 instead. Given the same crash, we see a stacktrace looking like Listing 4. Notice that it contains all the intervening calls to `fn1`, and in a debugger, we can freely inspect all of this context. Our context window becomes a complete, explorable description of how the program got to where it failed.

Anecdotally, the difference in convenience causes a significant design dilemma: whether to benefit from the flexibility of a control flow DSL, or to risk the bugs and code complexity of a design that does not use one. This should not need to be a choice – we should be able to do both.

3 A Way Forward

Our proposed solution is to implement a debugger specialized to our DSL, so it can offer a debugging experience comparable to plain Scala.

The above statement is known to be an obvious but impractical option, since it is a selling point of Scala DSLs that they are just libraries, without the burden of publishing or consuming custom language tooling. That is, however, assuming our debugger cannot also be a lightweight library – minimally invasive and requiring no additional tooling support.

We present a method for implementing the Debug Adapter Protocol (DAP) [11] in-process, running alongside the DSL

implementation in a separate thread. This separate thread implements a working DAP server, to which any compatible IDE can make requests.

To work with our adapter, a DSL should be instrumented with log-like *point of interest* statements, which it calls as it runs. Points of interest should report information relevant to the specific DSL implementation state, and should include file and line number information wherever possible. These can be gathered by adjusting DSL term construction to summon values from Li Haoyi’s sourcecode library [7], or another similar utility.

Beyond reporting DSL state and source code locations, point of interest statements also allow the debug adapter to transparently implement breakpoints, single-step execution, and other similar features. For example, given an existing request to pause execution, when a point of interest is invoked, it can block the calling DSL thread. This yields full control to the debugger, which can then inspect DSL state or resume execution as desired.

As of writing, we have built a prototype of our proposed debugger for our Forja DSL [6]. We rely on the `TailCalls` DSL for exposition, and we will ideally have a prototype that works with `TailCalls` too by the time we present. If not, we can demo our Forja debugger, and in either case, share what we’ve learned so far.

The goal is to demonstrate that our technique is possible, and open up discussion around this type of debugging experience for DSLs. All of our code is open source. It is currently part of the Forja codebase.

4 Related Work

We are aware of relatively little work on solving this problem. Some projects we could find target external DSL toolchains [4, 10], meaning that they rely on the DSL being implemented using a code generator. A debugger for GEMOC [2] has notable similarities to our design in its instrumentation philosophy, albeit with a focus on tracing model state changes. The Moldable Debugger [3] relies on the rare ability to programmatically interfere with how a program’s statements execute.

5 Talk Outline

In this section, we sketch the proposed 20 minute talk’s contents, with point-wise time estimates. Indented time estimates are sub-points, and sum to the outer time estimate.

2min Introduction. Brief background on presenters, identify control flow DSL debugging topic.

4min Motivation: describe Section 2. Discuss Listings 1 and 2, highlight differences between Listings 3 and 4.

5min Demo: debugging Listing 2. Show exception breakpoint, single-stepping, inspecting the virtual callstack. Note: this is aspirational, but we will fall back to a Forja DSL example in the worst case.

¹Even if we got lucky and `fn2` threw directly (as opposed to in a `tailcall` thunk, which we do to make the issue clearer), we would gain one additional stack entry indicating `fn1`. In any large application with more than 2 methods calling each other, that isn’t much better.

9min How is it done? Discuss Section 3.

2min Design diagram.

- Identify normal DSL operation.
- DSL nodes have embedded source location (from sourcecode macros).
- Separate thread serving Debug Adapter Protocol (DAP) requests.
- Identify points of interest as communication primitive.

4min How to look like a real debugger.

- Accessing DSL source locations with sourcecode.
- Inferring a virtual call stack for TailCalls, based on DSL execution.
- DAP has a lot of fallbacks. Impossible to implement features can be ignored.

3min Pausing the program is thread synchronization.

- Locking inside a point of interest is functionally thread pausing.
- Sufficiently frequent points of interest can avoid being unable to stop the DSL, since we can't truly pre-empt.
- Locking out the DSL means we get thread-safe access to DSL data structures, if we need them.

1min Wrap-up.

- We showed how to get a usable debugging experience for a control flow DSL.
- While it requires work to adopt, many parts of our DAP server are reusable.
- Discussion: should we do more of this? How should this technique be evaluated?

References

- [1] Rúnar Óli Bjarnarson. 2012. Stackless scala with free monads. *Scala Days* (2012).
- [2] Erwan Bousse, Dorian Leroy, Benoit Combemale, Manuel Wimmer, and Benoit Baudry. 2018. Omniscient debugging for executable DSLs. *Journal of Systems and Software* 137 (2018), 261–288. doi:10.1016/j.jss.2017.11.025
- [3] Andrei Chiş, Tudor Girba, and Oscar Nierstrasz. 2014. The Moldable Debugger: A Framework for Developing Domain-Specific Debuggers. In *Software Language Engineering*, Benoit Combemale, David J. Pearce, Olivier Barais, and Jurgen J. Vinju (Eds.). Springer International Publishing, Cham, 102–121.
- [4] Josselin Enet, Erwan Bousse, Massimo Tisi, and Gerson Sunyé. 2023. Protocol-Based Interactive Debugging for Domain-Specific Languages. *Journal of Object Technology* 22, 2 (July 2023), 2:1–14. doi:10.5381/jot.2023.22.2.a6 The 19th European Conference on Modelling Foundations and Applications (ECMFA 2023).
- [5] EPFL and Lightbend. 2025. scala/scala-parser-combinators. <https://github.com/scala/scala-parser-combinators>.
- [6] Forja Team. 2025. DistCompiler/forja. <https://github.com/DistCompiler/forja/>.
- [7] Li Haoyi. 2025. com-lihaoyi/sourcecode. <https://github.com/com-lihaoyi/sourcecode>.
- [8] Li Haoyi. 2025. FastParse 3.1.1. <https://com-lihaoyi.github.io/fastparse/>.
- [9] LAMP/EPFL. 2025. TailCalls. [https://www.scala-lang.org/api/current/scala/util/control/TailCalls\\$.html](https://www.scala-lang.org/api/current/scala/util/control/TailCalls$.html).
- [10] Ricky T. Lindeman, Lennart C.L. Kats, and Eelco Visser. 2011. Declaratively defining domain-specific language debuggers. In *Proceedings of the 10th ACM International Conference on Generative Programming and Component Engineering* (Portland, Oregon, USA) (GPCE '11). Association for Computing Machinery, New York, NY, USA, 127–136. doi:10.1145/2047862.2047885
- [11] Microsoft. 2025. Official Page for Debug Adapter Protocol. <https://microsoft.github.io/debug-adapter-protocol/>.
- [12] TypeLevel. 2025. Cats Effect, The pure asynchronous runtime for Scala. <https://typelevel.org/cats-effect/>.
- [13] TypeLevel. 2025. Eval. <https://typelevel.org/cats/datatypes/eval.html>.
- [14] Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauly, Michael J Franklin, Scott Shenker, and Ion Stoica. 2012. Resilient distributed datasets: A Fault-Tolerant abstraction for In-Memory cluster computing. In *9th USENIX symposium on networked systems design and implementation (NSDI 12)*. 15–28.
- [15] ZIO Maintainers. 2025. ZIO. <https://zio.dev/>.